Operating Systems + Algorithms

🌟 PART 1 — What Is an Operating System?

The operating system (OS) is the main software that controls how your computer works. It acts like a manager between the hardware (CPU, RAM, disk, keyboard, screen…) and all the programs you run. Without an OS, your applications wouldn't know how to use your hardware. When you click an app, the OS loads it into memory, gives it CPU time, allows it to read/write files, and manages all the background tasks. Examples are Windows, Linux, Android, iOS. The OS makes sure everything works smoothly without conflict — just like a traffic controller making sure cars don't crash into each other.

🌟 PART 2 — Processes & Threads (Explanation Paragraph)

A **process** is simply a program that is currently running. When you open a browser, watch a video, or open Word, each one becomes a separate process handled by the OS. A **thread** is a smaller worker inside a process — for example, in Chrome, one thread loads the page, another plays the video, another handles clicks. Processes are like offices in a building, and threads are workers inside each office. The OS decides when each process or thread gets CPU time and ensures they don't block or harm each other.

▬▬▬▬▬▬▬▬▬▬▬▬▬▬▬▬▬▬▬▬▬▬▬

🌟 PART 3 — Memory Management (Explanation Paragraph)

▬▬▬▬▬▬▬▬▬▬▬▬▬▬▬▬▬▬▬▬▬▬▬

When a program starts, the OS gives it space in **RAM** so it can run fast. RAM is limited, so the OS must constantly decide which process gets how much space, when to free memory, and how to move inactive parts to **virtual memory** on disk. If a program uses too much memory, it can crash ("stack overflow" or "out of memory"). The OS also protects memory: one program cannot access another program's memory — this is how the OS guarantees safety. Memory management is one of the OS's most complex responsibilities.

▬▬▬▬▬▬▬▬▬▬▬▬▬▬▬▬▬▬▬▬▬▬▬

🌟 PART 4 — File System (Explanation Paragraph)

▬▬▬▬▬▬▬▬▬▬▬▬▬▬▬▬▬▬▬▬▬▬▬

The **file system** is how the OS organizes and stores your files. It manages folders, file names, extensions, and permissions. When you save a photo, the OS decides where on the disk to place it, how to index it, and how to retrieve it later. The OS also controls file permissions: who is allowed to read a file, edit it, or execute it. Without a file system, data would be just random bytes on a disk with no organization.

▬▬▬▬▬▬▬▬▬▬▬▬▬▬▬▬▬▬▬▬▬▬▬

🌟 PART 5 — Scheduling Algorithms (FIFO, LIFO, SJF, Priority)

Explain these slowly and with simple real-life analogies.

⭐ 1. FIFO – First In, First Out

Explanation paragraph:
FIFO means the first process that enters the queue is the first one to be served. Imagine people waiting in line at a bakery. Whoever came first will be served first — simple, fair, and predictable. In OS scheduling, FIFO is used when processes arrive and are handled in order of arrival. It's easy to implement but can be slow if the first process takes too long.

⭐ 2. LIFO – Last In, First Out

Explanation paragraph:
LIFO means the last process that entered the queue is the first to be executed. Think of stacking plates: you always take the last plate added on top. In OS scheduling, LIFO is rarely used because new tasks would always "jump in front," and older ones could starve forever. But the idea is useful to understand stack behavior in memory and some CPU operations.

⭐ 3. SJF – Shortest Job First

Explanation paragraph:
SJF means the OS picks the process that needs the least amount of CPU time. This is like serving the customer who needs only one small item before serving someone with a huge list. It makes the average waiting time much shorter and is optimal in theory — but the OS doesn't always know how long a job will take. It's great when tasks have predictable lengths.

⭐ 4. Priority Scheduling

Explanation paragraph:
In Priority Scheduling, every process is assigned a priority — high or low. The OS always runs the process with the highest priority first. This is like an emergency patient in a hospital: even if many patients were waiting before, a critical case gets treated first. The downside is **starvation** — low-priority tasks might never run if high-priority ones arrive constantly. OS systems fix this by gradually increasing the priority of waiting tasks.

Open Task Manager and show:

- Processes tab
- CPU % and RAM %
- How closing a process stops it immediately
- Sorting processes to show which are using most CPU (SJF logic)
- Show priorities (Right click → Set Priority)

This will connect everything.

▬▬▬▬▬▬▬▬▬▬▬▬▬▬▬▬▬▬▬▬▬▬▬

🌟 PART 7 — Simple Beginner Algorithms Exercises

▬▬▬▬▬▬▬▬▬▬▬▬▬▬▬▬▬▬▬▬▬▬▬

Here are **very beginner-level** exercises that are NOT duplicates of calculator/string basics, and still build thinking:

Exercise 1 — Count Occurrences of an Element in a List

Problem:
Write a function that takes a list (array) and a number, and returns how many times that number appears in the list.

Example:
Input: list = [2, 3, 2, 5, 2], number = 2
Output: 3

Exercise 2 — Check If List Is Sorted (Ascending)

Problem:
Write a function that checks if a list is sorted in increasing order.
Return `True` if sorted, otherwise `False`.

Example:
[1, 2, 3, 4] → True
[1, 3, 2] → False

Exercise 3 — Find the Smallest Number in a List

Problem:
Write a function that takes a list of numbers and returns the smallest one.

Example:
Input: [5, 2, 8, 1, 9]
Output: 1

Exercise 4 — Count How Many Numbers Are Even

Problem:
Write a function that takes a list and returns how many numbers are even.

Example:
Input: [2, 4, 7, 9, 10]
Output: 3

----

in this document you gained a tiny knowledge about how an operating system manages programs, memory, and files.

You saw how the OS decides which task gets the CPU using FIFO, LIFO, SJF, and priority scheduling.

And we practiced algorithms to strengthen your problem-solving.